

Software Requirements Document (SRD/SRS)

Cyber Fraud Reporting and Prevention Portal for Defence Personnel

Project Type: Major Project

Institute: Maharashtra Institute of Technology, Chh. Sambhajinagar

Team: Rushikesh Ganesh Mundhe, Shekhar Gajanan Tayde, Ashish Prashant Vaidya

Guide: Mr. R. N. Patil Sir

1. Document Purpose

This Software Requirements Document defines the complete functional and technical requirements for the **Cyber Fraud Reporting and Prevention Portal for Defence Personnel**.

The document is intended to act as a common reference for:

- Frontend developers
- Backend developers
- ML developers
- Database developers
- Security developers
- Admin/CERT-Army dashboard developers
- Testing team
- Integration team

The main objective is to ensure that all teams develop against the **same API contracts, database schema, ML input/output format, authentication mechanism, and workflow**.

The proposed system is a secure web platform through which defence personnel, families, and veterans can report cyber threats, upload evidence, check suspicious URLs/emails/numbers, receive preventive guidance, and track complaints. The system also provides automated threat classification and incident prioritization.

2. Problem Statement

Defence personnel may encounter phishing messages, fraudulent calls, malicious links, malware, impersonation attempts, espionage-related threats, and OPSEC-related risks.

The project research identifies several problems:

- No dedicated cyber-fraud reporting platform specifically designed for defence personnel.

- Delays in reporting incidents.
- Difficulty tracking complaints.
- Limited awareness of official reporting procedures.
- Lack of specialized security support for defence users.
- Lack of integrated preventive tools.

The proposed system addresses these issues through a centralized, secure reporting and prevention platform.

3. Project Objectives

The system shall:

1. Allow defence personnel to securely report cyber fraud.
2. Allow users to upload supporting evidence.
3. Provide complaint tracking.
4. Provide suspicious URL/email analysis.
5. Provide spam-number reporting/checking.
6. Automatically classify cyber threats using ML.
7. Prioritize incidents according to severity and urgency.
8. Provide security recommendations to users.
9. Provide an administrative/CERT-Army dashboard.
10. Support offline complaint drafting and evidence storage.
11. Synchronize offline data when connectivity is restored.
12. Maintain evidence integrity using cryptographic hashes.
13. Maintain complete audit trails.
14. Provide cybersecurity awareness information.

These requirements directly follow the objectives and unique-value propositions defined in the project presentation.

4. System Users

4.1 Defence User

A defence personnel, family member, or veteran using the system to:

- Report incidents
- Upload evidence
- Check suspicious content
- Track complaints
- Receive alerts
- Read awareness material

- Work offline when required

4.2 Investigator / CERT-Army Officer

Responsible for:

- Viewing assigned incidents
- Reviewing evidence
- Reviewing ML predictions
- Updating case status
- Adding investigation notes
- Assigning cases
- Escalating critical incidents

4.3 Administrator

Responsible for:

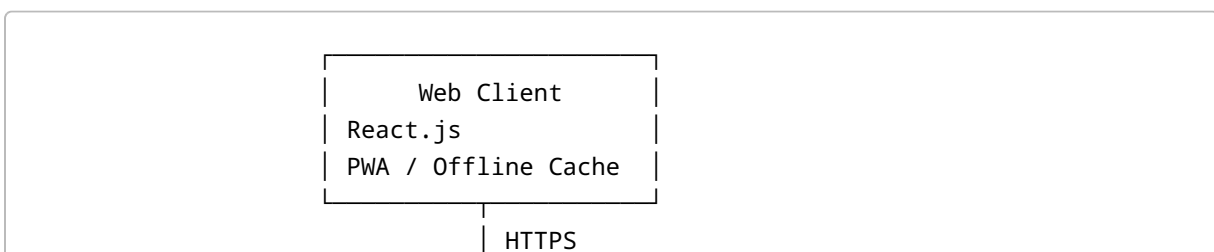
- User management
- Role management
- Threat-category management
- Dashboard monitoring
- ML monitoring
- Audit logs
- System configuration

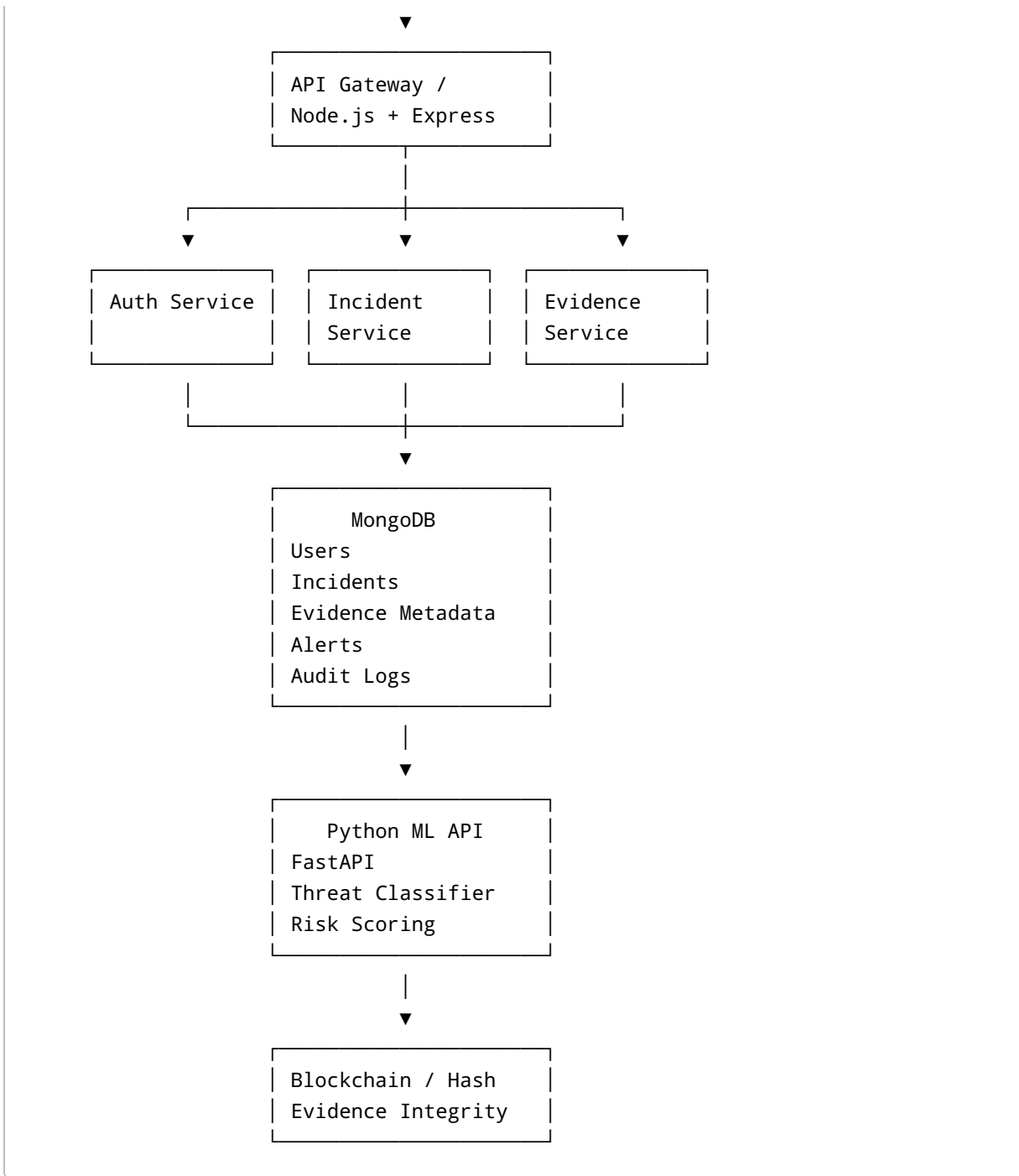
4.4 ML Service

Responsible for:

- Threat classification
- Suspicious-content analysis
- Risk scoring
- Confidence calculation
- Feature extraction

5. High-Level System Architecture





6. Recommended Technology Stack

Frontend

- React.js

- React Router
- Tailwind CSS
- Axios
- React Query / TanStack Query
- Context API or Redux Toolkit
- IndexedDB
- Service Worker
- PWA functionality
- WebSocket/Socket.IO for real-time updates

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- bcrypt/Argon2
- Socket.IO
- Multer or equivalent upload middleware
- Redis where required
- REST API

The project presentation explicitly identifies React.js for the web interface and Node.js + Express.js + MongoDB for backend logic.

ML

- Python
- FastAPI
- scikit-learn
- pandas
- NumPy
- joblib
- Optional NLP libraries

The project presentation identifies Python as the AI/ML technology for threat classification, fake-report filtering, and malware-related analysis.

Security

- HTTPS/TLS
- AES-256
- JWT
- Role-Based Access Control
- Password hashing
- Audit logging

- Cryptographic hashing
 - Blockchain integrity records
-

7. Frontend Requirements

7.1 Public/Authentication Pages

Login

Fields:

```
serviceId  
password  
captcha  
rememberDevice
```

Actions:

- Login
 - Forgot password
 - Account recovery
-

Registration

Fields:

```
fullName  
serviceId  
email  
phone  
userType  
organization  
rank  
password  
confirmPassword  
securityQuestion  
securityAnswer
```

Possible `userType`:

```
DEFENCE_PERSONNEL  
FAMILY_MEMBER  
VETERAN
```

For real deployment, sensitive defence information should be minimized and collected only when operationally necessary.

8. User Dashboard

Dashboard sections:

```
Total Reports  
Open Cases  
Under Investigation  
Resolved Cases  
Critical Alerts  
Recent Activity  
Threat Alerts  
Quick Actions
```

Quick actions:

- Report Cyber Fraud
 - Check URL
 - Check Email
 - Check Phone Number
 - Upload Evidence
 - Track Complaint
 - Cyber Awareness
-

9. Incident Reporting Module

Step 1 — Incident Information

Fields:

```
incidentType  
incidentDate  
incidentTime  
description
```

```
channel
financialLoss
lossAmount
currency
suspectedSource
```

Incident types:

```
PHISHING
FINANCIAL_FRAUD
MALWARE
IDENTITY_THEFT
SPAM
SOCIAL_ENGINEERING
ESPIONAGE
OPSEC_RISK
MALICIOUS_URL
FAKE_EMAIL
FAKE_CALL
OTHER
```

Step 2 — Evidence

Supported formats from the project concept:

- Text
- URL
- Image
- Audio
- Video
- File
- Screenshot
- Call log

Evidence metadata:

```
evidenceId
incidentId
uploadedBy
fileName
fileType
mimeType
fileSize
```

```
storagePath
sha256Hash
uploadedAt
sourceType
isEncrypted
blockchainTxId
```

10. Incident Submission Flow

```
User
↓
Create Incident
↓
Enter Incident Details
↓
Upload Evidence
↓
Backend validates input
↓
Evidence hash generated
↓
ML analysis
↓
Threat classification
↓
Risk score
↓
Severity calculation
↓
Incident created
↓
CERT/Admin queue
↓
User receives complaint ID
```

11. Complaint Status

Recommended statuses:

DRAFT
SUBMITTED
RECEIVED
AI_ANALYSIS
UNDER_REVIEW
ASSIGNED
UNDER_INVESTIGATION
ESCALATED
RESOLVED
CLOSED
REJECTED

Status transitions must be controlled by the backend.

12. Complaint Tracking Page

Display:

Complaint ID
Created Date
Incident Type
Current Status
Severity
Priority
Assigned Officer
Last Updated
Expected Action
Timeline

Example:

CRF-2026-000142

Submitted
↓
AI Analysis
↓
Under Review
↓
Assigned
↓
Under Investigation

↓
Resolved

13. Threat Analysis Tools

13.1 URL Checker

Input:

url

Output:

```
isMalicious
riskScore
classification
confidence
reasons
recommendation
```

Example:

```
{
  "classification": "PHISHING",
  "riskScore": 91,
  "confidence": 0.96,
  "recommendation": "Do not open the URL."
}
```

14. Email Checker

Input:

```
senderEmail
subject
body
```

```
headers
attachments
```

ML features can include:

- Suspicious keywords
- URL count
- Domain characteristics
- Sender-domain mismatch
- Urgency language
- Credential requests
- Attachment characteristics

Output:

```
classification
riskScore
confidence
indicators
recommendation
```

15. Phone/Spam Checker

Input:

```
phoneNumber
countryCode
message
callType
```

Output:

```
riskScore
spamProbability
classification
reportedCount
recommendation
```

16. Prevention & Awareness Module

Sections:

- Current threats
- Phishing awareness
- Password security
- Social engineering
- Safe browsing
- Mobile security
- OPSEC awareness
- Suspicious-call awareness

The objective is to improve cybersecurity awareness and reduce vulnerability through preventive information.

17. Real-Time Alerts

Users should receive alerts for:

```
Critical Threat
Case Status Change
Officer Assignment
Investigation Update
Security Warning
System Notification
```

Backend:

```
Node.js
  ↓
Socket.IO
  ↓
React Client
```

18. Offline Functionality

The project specifically requires offline complaint drafting and evidence storage with synchronization once connectivity is restored.

Frontend should use:

Service Worker
IndexedDB
Local Queue
Sync Manager

Offline record:

```
{
  "localId": "offline-uuid",
  "operation": "CREATE_INCIDENT",
  "payload": {},
  "createdAt": "...",
  "syncStatus": "PENDING"
}
```

When internet becomes available:

```
Offline Data
  ↓
Connectivity detected
  ↓
Sync Queue
  ↓
Authenticate
  ↓
POST to Backend
  ↓
Backend validates
  ↓
Create server record
  ↓
Return server incidentId
  ↓
Update local record
  ↓
SYNCED
```

19. Backend Architecture

Recommended backend structure:

```
server/  
|  
├─ src/  
|   ├─ config/  
|   ├─ controllers/  
|   ├─ models/  
|   ├─ routes/  
|   ├─ middleware/  
|   ├─ services/  
|   ├─ validators/  
|   ├─ utils/  
|   ├─ sockets/  
|   └─ app.js  
├─ uploads/  
├─ tests/  
└─ server.js
```

20. MongoDB Database Schema

20.1 Users Collection

```
{  
  _id,  
  fullName,  
  serviceId,  
  email,  
  phone,  
  userType,  
  organization,  
  rank,  
  passwordHash,  
  role,  
  isVerified,  
  isActive,  
  lastLoginAt,  
  createdAt,  
}
```

```
    updatedAt
  }
```

Roles:

```
USER
INVESTIGATOR
ADMIN
SUPER_ADMIN
```

21. Incident Schema

```
{
  _id,
  complaintId,
  userId,

  incidentType,
  title,
  description,

  incidentDate,
  incidentTime,

  channel,

  suspectedSource,

  financialLoss,
  lossAmount,
  currency,

  location,

  status,

  severity,
  priority,

  aiAnalysis: {
    classification,
    confidence,
    riskScore,
```

```
    modelVersion,  
    analyzedAt  
  },  
  
  assignedOfficerId,  
  
  createdAt,  
  updatedAt,  
  submittedAt,  
  resolvedAt  
}
```

22. Evidence Schema

```
{  
  _id,  
  incidentId,  
  uploadedBy,  
  
  fileName,  
  fileType,  
  mimeType,  
  fileSize,  
  
  storagePath,  
  
  sha256Hash,  
  
  encryptionAlgorithm,  
  
  blockchain: {  
    network,  
    transactionId,  
    blockNumber,  
    timestamp  
  },  
  
  uploadedAt  
}
```

The PPT specifically proposes storing cryptographic hashes on blockchain to protect evidence integrity.

23. AI Analysis Schema

```
{
  _id,

  incidentId,

  inputType,

  modelName,
  modelVersion,

  prediction: {
    label,
    confidence,
    riskScore
  },

  probabilities: {
    phishing,
    malware,
    spam,
    espionage,
    opsecRisk,
    financialFraud,
    socialEngineering,
    legitimate
  },

  featuresUsed: [],

  indicators: [],

  recommendation,

  analyzedAt
}
```

24. Audit Log Schema

Every sensitive operation should be logged.

```
{
  _id,

  actorId,
  actorRole,

  action,

  resourceType,
  resourceId,

  ipAddress,
  userAgent,

  metadata,

  timestamp
}
```

Examples:

```
LOGIN
LOGOUT
CREATE_INCIDENT
VIEW_INCIDENT
DOWNLOAD_EVIDENCE
UPLOAD_EVIDENCE
ASSIGN_CASE
CHANGE_STATUS
DELETE_RECORD
AI_ANALYSIS
ADMIN_ACTION
```

25. Notification Schema

```
{
  _id,
  userId,

  type,
  title,
  message,
}
```

```
    incidentId,  
  
    isRead,  
  
    createdAt  
}
```

26. Threat Intelligence Schema

```
{  
  _id,  
  
  indicatorType,  
  
  indicatorValue,  
  
  classification,  
  
  riskScore,  
  
  source,  
  
  confidence,  
  
  firstSeen,  
  lastSeen,  
  
  isActive,  
  
  createdAt  
}
```

`indicatorType` :

```
URL  
EMAIL  
PHONE  
IP  
DOMAIN  
FILE_HASH
```

27. API Specification

Authentication

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/logout
POST /api/v1/auth/refresh
GET /api/v1/auth/me
```

Incidents

```
POST /api/v1/incidents
GET /api/v1/incidents
GET /api/v1/incidents/:id
PATCH /api/v1/incidents/:id
DELETE /api/v1/incidents/:id
POST /api/v1/incidents/:id/submit
```

Evidence

```
POST /api/v1/incidents/:id/evidence
GET /api/v1/incidents/:id/evidence
GET /api/v1/evidence/:id
```

Threat Detection

```
POST /api/v1/analyze/url
POST /api/v1/analyze/email
POST /api/v1/analyze/phone
POST /api/v1/analyze/file
POST /api/v1/analyze/incident
```

Case Management

```
GET    /api/v1/cases
GET    /api/v1/cases/:id
PATCH /api/v1/cases/:id/status
PATCH /api/v1/cases/:id/assign
POST   /api/v1/cases/:id/notes
```

Notifications

```
GET    /api/v1/notifications
PATCH /api/v1/notifications/:id/read
```

28. ML Model Architecture

The ML component should be implemented as a **separate Python service** rather than embedding ML directly inside the Node.js server.

Recommended:

```
React
↓
Node.js API
↓
Python FastAPI
↓
ML Model
↓
Prediction
↓
Node.js
↓
MongoDB
```

This separation allows the ML team to independently train, test, and replace models without changing the frontend.

29. ML Threat Classification

The model should classify threats into categories such as:

```
PHISHING
MALWARE
SPAM
FINANCIAL_FRAUD
SOCIAL_ENGINEERING
ESPIONAGE
OPSEC_RISK
LEGITIMATE
```

The project's presentation explicitly identifies phishing, malware, espionage, and OPSEC risks as target threat categories.

30. ML Input Contract

The Node backend should send a standardized JSON object to the Python ML service.

```
{
  "requestId": "uuid",
  "incidentId": "INC-123",
  "inputType": "URL",
  "text": "https://example.com/login",
  "metadata": {
    "source": "USER_REPORT",
    "timestamp": "2026-08-21T10:00:00Z"
  }
}
```

For email:

```
{
  "requestId": "uuid",
  "incidentId": "INC-123",
  "inputType": "EMAIL",
  "text": "Your account will be blocked...",
  "metadata": {
    "sender": "example@email.com",
    "subject": "Urgent verification"
  }
}
```

```
}  
}
```

31. ML Output Contract

This is one of the most important integration contracts.

The ML service **MUST** always return the same structure.

```
{  
  "requestId": "uuid",  
  "modelName": "cyber-threat-classifier",  
  "modelVersion": "1.0.0",  
  
  "prediction": {  
    "label": "PHISHING",  
    "confidence": 0.96,  
    "riskScore": 91  
  },  
  
  "probabilities": {  
    "PHISHING": 0.96,  
    "MALWARE": 0.01,  
    "SPAM": 0.01,  
    "FINANCIAL_FRAUD": 0.01,  
    "SOCIAL_ENGINEERING": 0.01  
  },  
  
  "indicators": [  
    "Urgent language",  
    "Credential request",  
    "Suspicious URL"  
  ],  
  
  "recommendation": "Do not open the URL or provide credentials.",  
  
  "processedAt": "2026-08-21T10:00:05Z"  
}
```

Frontend and backend developers should build against this contract.

32. ML Model Recommendation

For the first working version:

Text Classification

Use:

```
TF-IDF
+
Logistic Regression / Random Forest
```

For the project prototype this provides a comparatively simple, explainable baseline.

Possible future version:

```
Transformer / BERT-based classifier
```

URL Classification

Features:

```
URL length
Number of dots
Number of special characters
HTTPS presence
Domain length
Number of subdomains
IP address usage
Suspicious keywords
Entropy
Shortened URL
```

Risk Score

Normalize model probability into:

```
0 - 30    LOW
31 - 60   MEDIUM
61 - 80   HIGH
81 - 100  CRITICAL
```

This threshold scheme should be treated as a project configuration rather than hard-coded business logic.

33. Incident Prioritization Engine

ML classification and incident priority should be separate concepts.

Example:

```
Threat Type
+
ML Confidence
+
Financial Loss
+
Evidence
+
Target Sensitivity
+
Urgency
=
Priority Score
```

Example:

```
Priority Score: 92
Severity: CRITICAL
Priority: P1
```

Priority:

```
P1 = Critical
P2 = High
P3 = Medium
P4 = Low
```

The project specifically requires complaints to be ranked by severity and urgency so that critical threats reach the appropriate response team first.

34. CERT/Admin Dashboard

Dashboard modules:

Overview

Total Incidents
Critical Incidents
Pending Cases
Assigned Cases
Resolved Cases
Average Response Time

Case Queue

Columns:

Complaint ID
Type
Severity
Priority
Created At
Status
Assigned Officer

Case Details

Display:

User Information
Incident Information
Evidence
ML Prediction
Risk Score
Timeline
Officer Notes
Audit History

35. Investigator Workflow

```
New Complaint
  ↓
AI Analysis
  ↓
Priority Assignment
  ↓
Investigator Queue
  ↓
Officer Assignment
  ↓
Evidence Review
  ↓
Investigation
  ↓
Status Update
  ↓
Resolution
  ↓
Case Closure
```

36. Blockchain Integration

Blockchain should NOT store the actual evidence file.

Instead:

```
Evidence File
  ↓
SHA-256 Hash
  ↓
Blockchain Record
```

Blockchain record:

```
{
  "evidenceId": "EV-123",
  "incidentId": "INC-123",
  "sha256Hash": "...",
```

```
"timestamp": "...",  
"transactionId": "..."  
}
```

During investigation:

```
Current File  
↓  
Generate SHA-256  
↓  
Compare with Blockchain Hash  
↓  
MATCH → Evidence unchanged  
MISMATCH → Evidence integrity violation
```

37. Security Requirements

The project specifies AES-256 and HTTPS/TLS 1.3 as security mechanisms.

Required security controls:

- HTTPS
 - Secure cookies/token handling
 - Password hashing
 - JWT access control
 - Role-based authorization
 - Input validation
 - Rate limiting
 - File type validation
 - File size limits
 - Malware scanning of uploads
 - Audit logs
 - Encryption at rest
 - Encryption in transit
 - Secure session management
 - CSRF protection where applicable
 - XSS protection
 - No sensitive data in logs
 - No secrets in source code
-

38. File Upload Security

The backend must never blindly trust uploaded files.

Validation:

```
Extension
+
MIME type
+
File signature
+
Size
+
Malware scan
```

Rejected examples:

```
Executable files
Unknown binary formats
Oversized files
Invalid MIME types
Corrupted files
```

39. Authentication Flow

```
User
↓
Login
↓
Backend validates credentials
↓
Password verification
↓
Generate access token
↓
Generate refresh token
↓
Frontend authenticated
```

↓
Protected API calls

Every protected endpoint must verify:

Authentication
+
Authorization
+
Resource ownership

40. Role-Based Access Control

Function	User	Investigator	Admin
Create Complaint	✓	✓	✓
Upload Evidence	✓	✓	✓
View Own Cases	✓	✓	✓
View Assigned Cases	—	✓	✓
Assign Case	—	✓/Admin	✓
Change Case Status	Limited	✓	✓
View All Cases	—	Limited	✓
User Management	—	—	✓
Audit Logs	—	Limited	✓
ML Monitoring	—	—	✓

41. Frontend-Backend-ML Integration Contract

This section should be treated as mandatory by the entire team.

Frontend → Backend

Frontend sends:

```
{
  "incidentType": "PHISHING",
  "title": "Suspicious message",
  "description": "Received a suspicious message",
  "incidentDate": "2026-08-21",
  "channel": "SMS"
}
```

Backend returns:

```
{
  "success": true,
  "data": {
    "incidentId": "INC-123",
    "complaintId": "CRF-2026-000123",
    "status": "SUBMITTED"
  }
}
```

Backend → ML

```
{
  "requestId": "REQ-123",
  "incidentId": "INC-123",
  "inputType": "TEXT",
  "text": "Your account will be blocked..."
}
```

ML → Backend

```
{
  "requestId": "REQ-123",
  "modelName": "cyber-threat-classifier",
  "modelVersion": "1.0.0",
  "prediction": {
    "label": "PHISHING",
    "confidence": 0.96,
    "riskScore": 91
  },
}
```

```
"indicators": [  
  "Urgent language",  
  "Credential request"  
]  
}
```

Backend → Frontend

```
{  
  "incidentId": "INC-123",  
  "status": "AI_ANALYSIS",  
  "severity": "CRITICAL",  
  "priority": "P1",  
  "aiAnalysis": {  
    "classification": "PHISHING",  
    "confidence": 0.96,  
    "riskScore": 91  
  }  
}
```

42. API Error Format

Every API should use the same error structure.

```
{  
  "success": false,  
  "error": {  
    "code": "VALIDATION_ERROR",  
    "message": "Invalid incident data",  
    "details": []  
  },  
  "requestId": "REQ-123"  
}
```

Standard error codes:

```
AUTH_REQUIRED  
FORBIDDEN  
VALIDATION_ERROR
```

```
RESOURCE_NOT_FOUND
DUPLICATE_RESOURCE
FILE_TOO_LARGE
UNSUPPORTED_FILE
ML_SERVICE_ERROR
DATABASE_ERROR
RATE_LIMITED
INTERNAL_ERROR
```

43. ML Failure Handling

The application must NOT fail if the ML service is temporarily unavailable.

Correct workflow:

```
Incident Submitted
    ↓
ML Service unavailable
    ↓
Store Incident
    ↓
status = AI_ANALYSIS_PENDING
    ↓
Retry ML request
    ↓
Prediction received
    ↓
Update Incident
```

This is important because ML is an assisting component; it should not become a single point of failure for complaint reporting.

44. Offline Synchronization Conflict Handling

Each offline record should have:

```
localId
clientCreatedAt
clientUpdatedAt
```

```
syncStatus
version
```

Statuses:

```
PENDING
SYNCING
SYNCED
FAILED
CONFLICT
```

For a conflict:

```
Offline Version
  +
Server Version
  ↓
Conflict Detection
  ↓
Backend Resolution
  ↓
Audit Log
```

45. Testing Requirements

Frontend Testing

Test:

- Login
- Registration
- Forms
- Validation
- File upload
- Dashboard
- Complaint tracking
- Offline mode
- Synchronization
- Responsive design

Backend Testing

Test:

- Authentication
- Authorization
- API validation
- Database operations
- File uploads
- ML integration
- Rate limiting
- Error handling

ML Testing

Metrics:

```
Accuracy
Precision
Recall
F1 Score
Confusion Matrix
ROC-AUC
```

For security classification, do not rely only on accuracy; false negatives should receive particular attention.

46. Integration Testing

The complete integration test should be:

```
React
↓
POST /incidents
↓
Node.js
↓
MongoDB
↓
Python ML API
↓
Prediction
↓
```

```
Node.js
↓
MongoDB Update
↓
Socket.IO Notification
↓
React Dashboard
```

Expected result:

```
Complaint Created
+
AI Prediction Stored
+
Severity Assigned
+
User Notified
+
Admin Queue Updated
```

47. Suggested Git Repository Structure

```
cyber-fraud-portal/
|
├── frontend/
|
├── backend/
|
├── ml-service/
|
├── docs/
|
├── database/
|
├── blockchain/
|
└── README.md
```

48. Team Division

Member 1 — Frontend

Responsible for:

```
React
UI/UX
Dashboard
Forms
Complaint Tracking
Evidence Upload UI
Offline PWA
Notifications
```

Member 2 — Backend

Responsible for:

```
Node.js
Express
MongoDB
Authentication
REST APIs
File Handling
Case Management
Socket.IO
Security
```

Member 3 — ML

Responsible for:

```
Dataset
Data preprocessing
Feature engineering
Model training
Evaluation
Threat classification
Risk scoring
FastAPI
```

```
ML API
Model versioning
```

If additional team members become available, blockchain/security and testing should be separated from backend.

49. Shared Development Contract

Before coding, the team should freeze these files:

```
/api-spec.yaml
/ml-contract.json
/database-schema.md
/error-codes.md
/auth-spec.md
```

Nobody should independently change API field names without informing the entire team.

For example, do NOT allow:

```
riskScore
risk_score
risk
score
```

to represent the same field.

The project should standardize on:

```
riskScore
```

50. Environment Variables

Backend

```
PORT=
MONGO_URI=
```

```
JWT_SECRET=  
JWT_REFRESH_SECRET=  
ML_SERVICE_URL=  
STORAGE_URL=  
ENCRYPTION_KEY=  
BLOCKCHAIN_RPC_URL=  
BLOCKCHAIN_CONTRACT_ADDRESS=
```

ML

```
PORT=  
MODEL_PATH=  
MODEL_VERSION=  
LOG_LEVEL=
```

Frontend

```
VITE_API_BASE_URL=  
VITE_SOCKET_URL=
```

Secrets must never be committed to Git.

51. Definition of Done

A module is considered complete only when:

- Frontend implemented
 - Backend API implemented
 - Database schema implemented
 - Validation implemented
 - Authentication/authorization implemented
 - Error handling implemented
 - Unit tests written
 - Integration tested
 - API documented
 - Security checked
 - Git branch merged
-

52. Minimum Viable Product

The first working version should contain:

Phase 1

Authentication
+
User Dashboard
+
Complaint Creation
+
Evidence Upload
+
Complaint Tracking

Phase 2

ML Threat Classification
+
URL Checker
+
Email Checker
+
Risk Score
+
Incident Prioritization

Phase 3

Admin Dashboard
+
Investigator Workflow
+
Notifications
+
Audit Logs

Phase 4

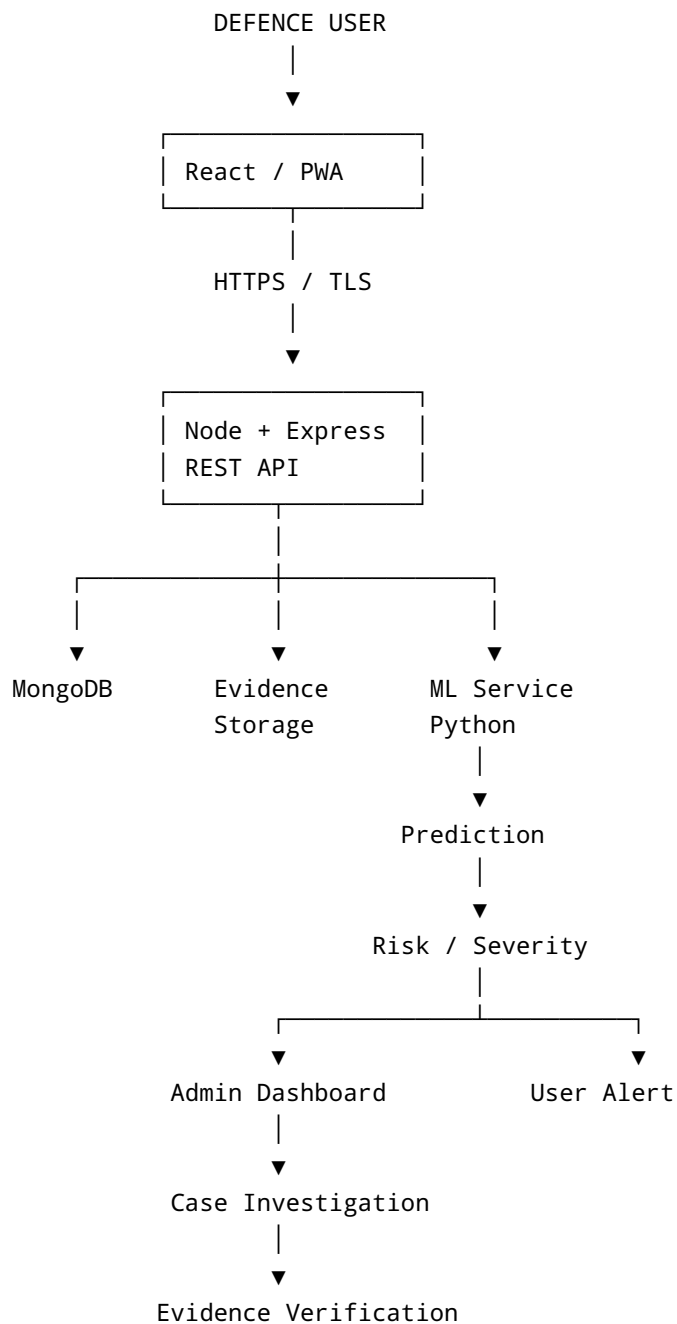
Offline Mode
+

53. Recommended Development Sequence

1. Freeze requirements
↓
2. Design database
↓
3. Define API contracts
↓
4. Define ML input/output contract
↓
5. Backend authentication
↓
6. Frontend authentication
↓
7. Incident APIs
↓
8. Incident UI
↓
9. Evidence APIs
↓
10. Evidence UI
↓
11. ML service
↓
12. Integrate ML
↓
13. Admin dashboard
↓
14. Real-time notifications
↓
15. Offline functionality
↓
16. Blockchain integrity
↓
17. Security testing
↓
18. Final integration

54. Final Integration Architecture

The final system should work as:



Blockchain Hash

55. Critical Integration Rules

The team should follow these rules throughout development:

1. **Frontend never directly communicates with MongoDB.**
2. **Frontend communicates only through backend APIs.**
3. **Backend communicates with ML through a defined API contract.**
4. **ML never directly modifies MongoDB.**
5. **Backend is responsible for storing ML predictions.**
6. **ML model version must be stored with every prediction.**
7. **Every incident must have a unique complaint ID.**
8. **Every evidence item must have a SHA-256 hash.**
9. **Every sensitive operation must generate an audit log.**
10. **Offline records must contain a client-generated UUID.**
11. **API error responses must follow one common format.**
12. **Authentication and authorization must be handled by the backend.**
13. **No secrets should be stored in frontend code.**
14. **ML failure must not prevent complaint submission.**
15. **API/database/ML field names must remain synchronized.**

56. Most Important Shared Data Flow

The complete integration contract can be summarized as:

```
USER
↓
React Form
↓
POST /api/v1/incidents
↓
Node/Express Validation
↓
MongoDB Incident
↓
ML Request
↓
Python FastAPI
```

```
↓  
Threat Prediction  
↓  
Risk Score  
↓  
Backend stores AI result  
↓  
Priority Engine  
↓  
Incident Queue  
↓  
CERT/Admin  
↓  
Investigation  
↓  
Resolution  
↓  
User Notification
```

This architecture keeps the three major development areas—**Frontend, Backend, and ML**—loosely coupled while keeping their data contracts synchronized.

57. Final Recommendation

For the project implementation, the most important document to maintain alongside this SRD is an **API Contract**. Before the frontend and ML teams start independently developing, freeze:

1. Database field names
2. API endpoint names
3. Request JSON
4. Response JSON
5. Authentication format
6. Error format
7. ML request format
8. ML response format
9. Incident statuses
10. Threat categories

This will prevent the common integration problem where the frontend expects one field, the backend returns another, and the ML service uses a third naming convention.

The uploaded project already establishes the core concept: secure reporting, evidence submission, AI/ML threat detection, incident prioritization, offline synchronization, and tamper-proof evidence handling. The

SRD above converts those project-level requirements into a team-oriented technical specification that can be used as the baseline for implementation.